

多周期 CPU 与五级流水线 CPU 改进实验报告

2026 年 6 月 9 日

目录

1	多周期 CPU 的 Bug 分析与修复	2
1.1	问题描述	2
1.2	修复方案	2
1.3	修复过程与验证	2
2	五级流水线 CPU 的 Bug 分析与修复	2
2.1	问题描述	2
2.2	修复方案	3
2.3	修复过程与验证	3
3	流水线 CPU 执行斐波那契数列计算任务	3
3.1	任务设计目标	3
3.2	MIPS 汇编程序及注释	3

1 多周期 CPU 的 Bug 分析与修复

1.1 问题描述

在多周期 CPU 实验中，当程序执行到地址 0x14 处的 `bgez` (Branch on Greater than or Equal to Zero) 指令时，CPU 出现分支错误，未能正确跳转到目标地址。经分析，这是由于 `bgez` 指令在取指 (IF) 阶段与访存 (MEM) 阶段之间存在 IP 核 (指令存储器) 输出延迟，导致译码阶段获得的分支目标地址计算不及时，从而使分支条件判断错误。

1.2 修复方案

在不改变硬件数据通路的前提下，最简洁的修复方法是在 `bgez` 指令之前插入一条 NOP 指令 (机器码 0x00000000)。NOP 指令会使流水线空转一个周期，给予 IP 核足够的响应时间，确保后续 `bgez` 的指令字及时进入译码阶段，从而正确计算分支地址并更新 PC。

1.3 修复过程与验证

1. 打开多周期 CPU 工程中的指令存储器初始化文件 (.coe)，在 0x14 地址处的 `bgez` 指令之前插入一行 0x00000000。
2. 将后续指令地址顺延，更新分支目标地址使其指向正确的指令行。
3. 重新综合、实现并生成比特流，下载到 FPGA。
4. 观察上板结果，确认 `bgez` 指令的跳转信号与 PC 值更新符合预期，程序继续正常执行。

经修复后，多周期 CPU 所有测试程序均通过验证，不再出现分支错误。

2 五级流水线 CPU 的 Bug 分析与修复

2.1 问题描述

在原始五级流水线 CPU 代码中，当执行以下指令序列时出现错误：

```
1 38H: sll $2, $1, 4
2 3CH: addu $3, $2, $1
```

Listing 1: 存在数据冲突的指令序列

指令 `addu` 的源操作数 `$2` 依赖于上一条指令 `sll` 的目的寄存器 `$2`。在流水线中，当 `addu` 进入译码阶段读取 `$2` 时，`sll` 指令尚处于执行阶段，其计算结果尚未写回寄存器堆，因此 `addu` 读到了 `$2` 的旧值，导致 `addu` 计算错误。这是典型的 RAW（Read After Write）数据相关。

2.2 修复方案

采用插入 NOP 指令的软件方法解决该数据冲突。在 `sll` 和 `addu` 之间插入两条 NOP 指令，使 `addu` 的译码阶段延迟一个周期，此时 `sll` 已完成写回，`$t0` 的新值可以被正确读取。

2.3 修复过程与验证

1. 修改指令存储器 `coe` 文件，在 `0x38` 和 `0x3C` 地址之间插入两条 `0x00000000`。
2. 调整后续指令地址，并重新编译。
3. 通过上板验证观察 `addu` 指令执行阶段的操作数，确认其读取的 `$2` 为移位后的新值。
4. 验证最终结果寄存器的值与预期结果一致。

修复后，流水线 CPU 能够正确执行原本存在冲突的指令序列，计算结果无误。

3 流水线 CPU 执行斐波那契数列计算任务

3.1 任务设计目标

编写一个无限循环计算斐波那契数列的 MIPS 汇编程序，并将其转换为机器码存入 `coe` 文件，交由五级流水线 CPU 执行。要求程序能够正确应对流水线中的数据相关，通过插入 NOP 指令保证计算结果的正确性。

3.2 MIPS 汇编程序及注释

程序设计为使用寄存器 `$t0`、`$t1` 和 `$t2` 分别保存数列的连续两项及下一项。初始值 $F(1)=1$ ， $F(2)=1$ ，然后不断迭代计算 $F(n)=F(n-1)+F(n-2)$ 。

以下是完整的汇编代码与对应的机器码。

1	0x00:	0x24080001	<code>addiu \$t0, \$zero, 1</code>	# <code>\$t0 = 1</code>	(<code>F(1)</code>)
2	0x04:	0x24090001	<code>addiu \$t1, \$zero, 1</code>	# <code>\$t1 = 1</code>	(<code>F(2)</code>)
3	#	----- 循环体开始 -----			

```

4 0x08: | 0x01095021 | addu $t2, $t0, $t1      # $t2 = $t0 + $t1 (F(n)
   )
5 0x0C: | 0x00000000 | nop                    # 延迟槽/解决数据冲突:
6      # $t2值在写回前被下一条需$t2的指令使用, 需插入NOP。
7 0x10: | 0x00000000 | nop                    # 插入2条NOP保证$t2写
   回。
8 0x14: | 0x00000000 | nop
9 0x18: | 0x00094021 | addu $t0, $zero, $t1    # $t0 = $t1 (更新F(n-2)
   )
10 0x1C: | 0x00000000 | nop                    # $t0写回后无立即冲突,
   但为统一流水线可加NOP
11 0x20: | 0x00000000 | nop
12 0x24: | 0x000A4821 | addu $t1, $zero, $t2    # $t1 = $t2 (更新F(n-1)
   )
13 0x28: | 0x0800000F | j      0x08            # 跳转至循环体首地址(0
   x08)
14 0x2C: | 0x00000000 | nop                    # 跳转延迟槽
15 0x30: | 0x00000000 | nop

```

Listing 2: 斐波那契数列计算汇编代码

经过上板验证, 可以正常进行运算